

AUTHENTICATION WORKFLOW CAPTURE AND JWT CLAIM ANALYSIS

Login request capture, bearer token handling, JWT decoding, and claim review

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 3 - Authentication and JWT Analysis
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	03-authentication-and-jwt-analysis-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 3 - Authentication and JWT Analysis
Objective	Understand the authentication workflow, capture JWT-based authorization behaviour, and inspect token claims safely without exposing live credentials.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

JWT decoding is a normal analysis step, not an exploit by itself. The professional value is in identifying what the token contains, whether sensitive claims are exposed, and whether the token model supports later authorization tests.

2. Evidence Embedded in This PDF

Evidence 1: 02-postman-login-jwt-token.png

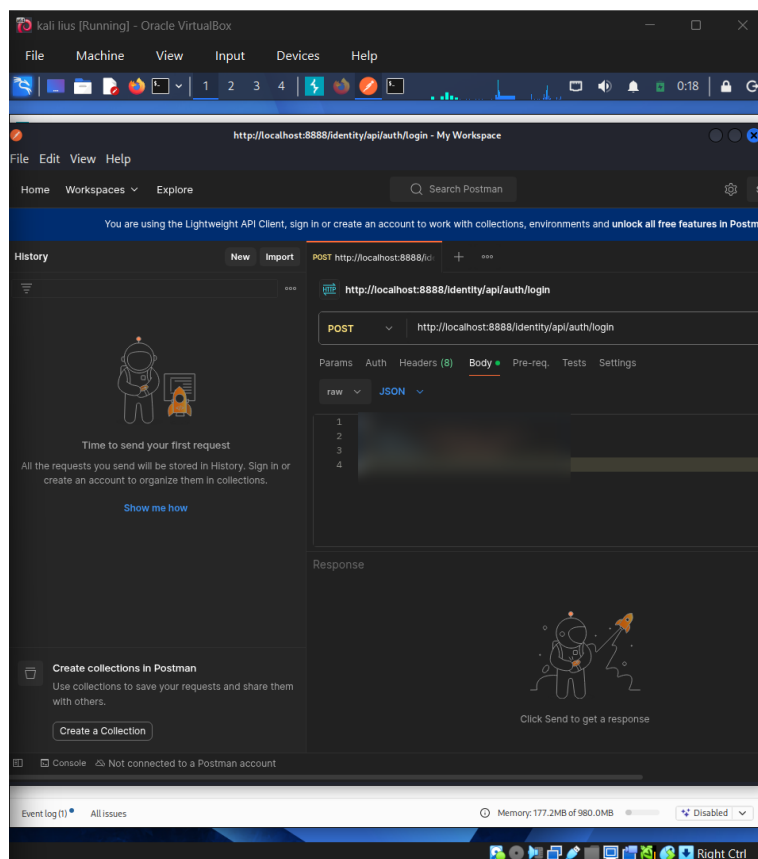


Figure 1 - Postman login evidence showing the authentication request and returned bearer token structure.

Postman login evidence showing the authentication request and returned bearer token structure.

Evidence 2: 03-jwtio-decoded-token.png

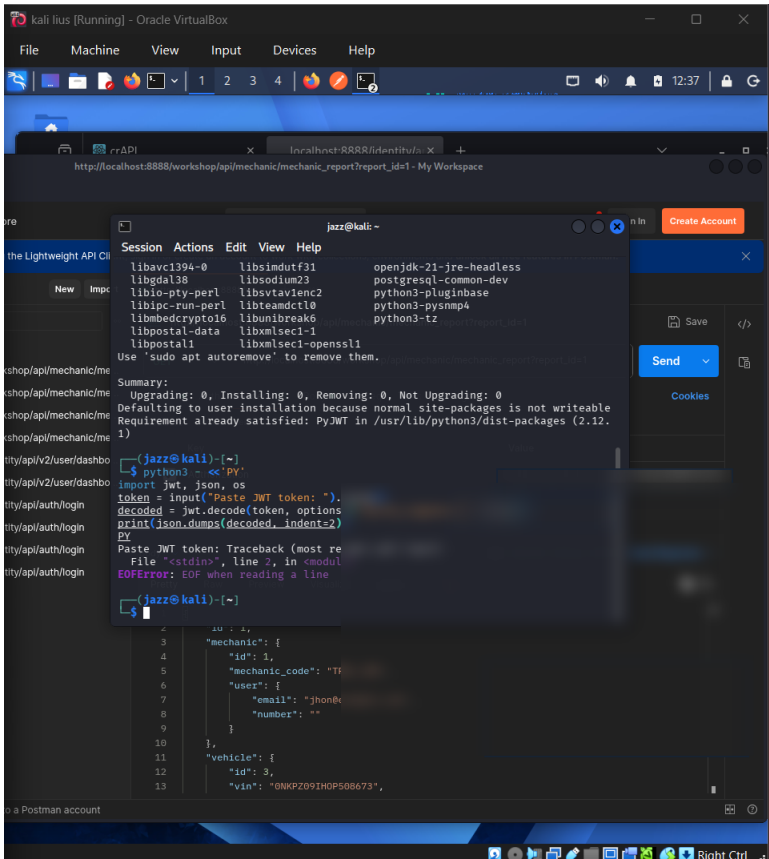


Figure 2 - JWT.io evidence showing decoded JWT header and payload for claim review.

JWT.io evidence showing decoded JWT header and payload for claim review.

Evidence 3: 06-JWT-decodetoken-analysis.png

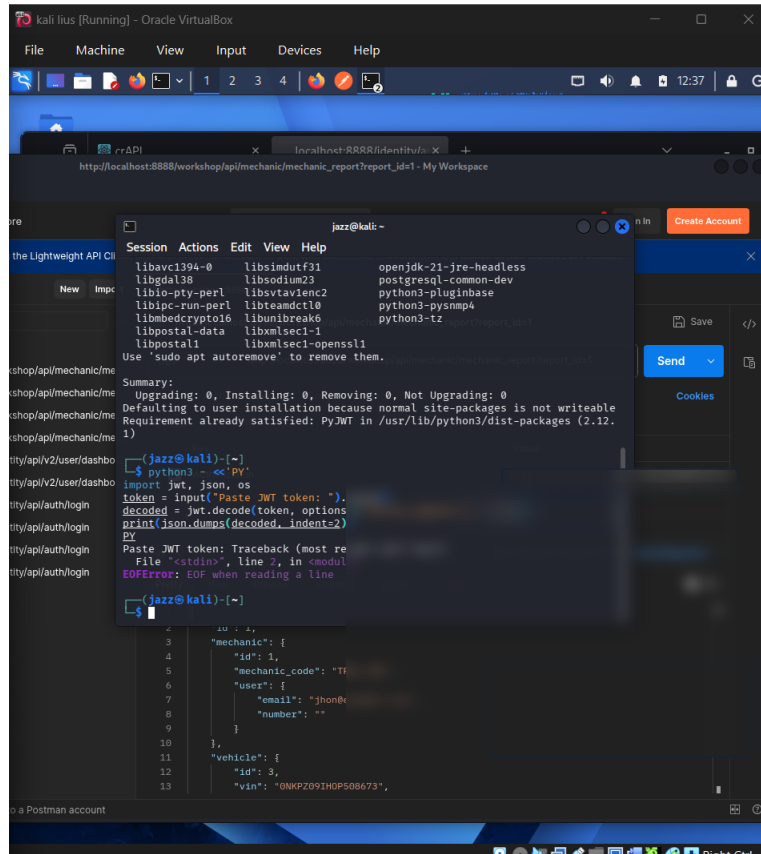


Figure 3 - Additional JWT claim review evidence showing local/token analysis workflow.

Additional JWT claim review evidence showing local/token analysis workflow.

3. Command and Workflow Used

```
POST /identity/api/auth/login
Content-Type: application/json
```

```
{
  "email": "lab-user@example.com",
  "password": "REDACTED"
}
```

```
Authorization: Bearer <REDACTED_TOKEN>
```

```
# Decode JWT locally without verifying signature for claim review only
python3 - <<'PY'
import jwt, json
token = "<REDACTED_TOKEN>"
print(json.dumps(jwt.decode(token, options={"verify_signature": False}), indent=2))
PY
```

Step-by-step workflow

- Captured the login request and response in Postman.
- Identified the bearer token returned by the authentication endpoint.
- Decoded the JWT header and payload for claim analysis only.
- Reviewed claims such as subject, role, issue time, expiry, and user identifiers.

- Redacted token values before publication.

4. Professional Interpretation

JWT decoding is a normal analysis step, not an exploit by itself. The professional value is in identifying what the token contains, whether sensitive claims are exposed, and whether the token model supports later authorization tests.

5. Security Risk and Impact

If JWTs contain sensitive data, weak signing, long expiry, missing audience/issuer validation, or trust client-controlled claims such as role or user ID, attackers may attempt privilege escalation or session abuse. Token values must be treated like credentials.

6. Recommended Remediation and Hardening

- Validate JWT signatures server-side on every protected request.
- Reject alg:none and unexpected algorithms.
- Validate issuer, audience, expiry, and not-before claims.
- Do not place sensitive personal data in JWT payloads.
- Use short token lifetimes and secure refresh-token handling.
- Protect signing secrets and rotate them if exposed.

7. Framework Mapping

Framework Area	Mapping
OWASP API	API2: Broken Authentication
Security Control	JWT validation and bearer token protection
Evidence Result	Authentication workflow captured and token claims reviewed

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.

- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.
- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.